

MazeGenerator — mini tutorial obsługi repozytorium

Dokument dla osoby, która chce pobrać, uruchomić, edytować lub opublikować projekt.

Cel dokumentu

Ten dokument prowadzi krok po kroku przez pobranie repozytorium, instalację zależności, uruchomienie projektu, edycję kodu, wykonanie commita oraz publikację zmian na GitHub Pages.

1. Wymagane narzędzia

- Node.js oraz npm — potrzebne do instalacji zależności i uruchomienia Vite.
- Git — potrzebny do pobrania repozytorium, commitowania i pushowania zmian.
- Edytor kodu, np. Visual Studio Code.
- Przeglądarka internetowa do testowania aplikacji lokalnie.

2. Pobranie repozytorium

W terminalu przejdź do folderu, w którym chcesz trzymać projekt, a następnie sklonuj repozytorium:

```
git clone https://github.com/ZeLussss/MazeGenerator.git
cd MazeGenerator
```

3. Instalacja zależności

Po pobraniu repozytorium zainstaluj paczki npm:

```
npm install
```

Nie commituj folderu node_modules. Folder ten powinien być ignorowany przez plik .gitignore.

4. Uruchomienie projektu lokalnie

Aby uruchomić aplikację developersko:

```
npm run dev
```

Vite pokaże lokalny adres, najczęściej:

```
http://localhost:5173/
```

5. Sprawdzenie typów i build

Przed commitem warto sprawdzić, czy TypeScript nie zgłasza błędów:

```
npm run typecheck
```

Następnie sprawdź build produkcyjny:

```
npm run build
```

Po buildzie Vite generuje folder dist. Tego folderu zwykle nie commitujemy, ponieważ GitHub Actions zbuduje projekt samodzielnie.

6. Najważniejsze pliki do edycji

Plik / folder	Do czego służy
src/core/Cell.ts	Model pojedynczej komórki labiryntu.
src/core/Wall.ts	Model ściany między dwiema komórkami.
src/maze/Maze.ts	Abstrakcyjna baza labiryntu, grid, ściany i Fisher-Yates.
src/maze/KruskalMaze.ts	Generator labiryntu algorytmem Kruskala.
src/solvers/AStarSolver.ts	Solver A* oraz kolejność odwiedzonych komórek.
src/solvers/DijkstraSolver.ts	Solver Dijkstry oraz kolejność odwiedzonych komórek.
src/render/CanvasRenderer.ts	Rysowanie labiryntu, ścieżki i odwiedzonych komórek.
src/frontend/main.ts	Logika przycisków, animacji i pobierania PNG.
src/frontend/style.css	Wygląd strony.
index.html	Struktura interfejsu użytkownika.

7. Typowy workflow pracy

```
git pull
npm install
npm run dev
# edycja kodu
npm run typecheck
npm run build
git status
git add .
git commit -m "Opis zmian"
git push
```

8. GitHub Pages

Projekt korzysta z Vite, więc na GitHub Pages powinien być publikowany zbudowany folder dist. Najwygodniej robić to przez GitHub Actions.

- W pliku vite.config.ts dla adresu <https://zelussss.github.io/MazeGenerator/> ustawiona jest baza: /MazeGenerator/.
- Workflow w .github/workflows/deploy.yml instaluje zależności, wykonuje npm run build i publikuje dist na GitHub Pages.
- W ustawieniach repozytorium GitHub Pages powinno mieć Source ustawione na GitHub Actions.

9. Najczęstsze problemy

Strona bez CSS na GitHub Pages

Najczęściej oznacza brak poprawnego builda Vite albo brak konfiguracji base w vite.config.ts.

Błąd npm ci w GitHub Actions

Najczęściej oznacza, że package.json i package-lock.json są niespójne. Uruchom lokalnie npm install, potem commituj oba pliki.

Błędy TypeScript

Uruchom `npm run typecheck` i popraw komunikaty. Szczególnie sprawdzaj importy, literówki w nazwach plików i typy metod.

10. Co commitować, a czego nie

- Commitować: `src`, `index.html`, `package.json`, `package-lock.json`, `tsconfig.json`, `vite.config.ts`, `README.md`, `.github/workflows`.
- Nie commitować: `node_modules`, `dist`, plików tymczasowych edytora, lokalnych plików `.env`.

Szybka komenda końcowa

Jeżeli wszystko działa: `npm run typecheck`, `npm run build`, `git add .`, `git commit -m "Update MazeGenerator"`, `git push`.